

Vamos con el **Sprint 6**.
Este sprint convierte QHALI en una herramienta más institucional: ya no solo reporta y valida, ahora permite **gestionar, filtrar, visualizar indicadores y detectar posibles duplicados**.

Sprint 6 — Dashboard inicial, filtros, cambio de estado y duplicados

Duración

1 semana: lunes a viernes

Objetivo del sprint

Implementar un dashboard inicial de gestión para visualizar incidentes, filtrar por estado y categoría, cambiar manualmente el estado de un incidente, mostrar indicadores básicos y detectar posibles reportes duplicados cercanos.

Este sprint es clave para que el MVP no parezca solo una app ciudadana, sino una plataforma inicial de gestión urbana.

Sprint Goal

Al finalizar el Sprint 6, QHALI debe permitir que un usuario administrador visualice incidentes en un dashboard, filtre reportes, actualice estados, consulte indicadores básicos y reciba advertencias de posible duplicidad al registrar incidentes cercanos similares.

Historias de usuario trabajadas en Sprint 6

Código	Historia de usuario	Prioridad
HU-26	Como administrador, quiero ver una lista de incidentes reportados para monitorear la carga de alertas urbanas.	P0
HU-27	Como administrador, quiero filtrar incidentes por estado para concentrarme en los reportes pendientes, confirmados o resueltos.	P0
HU-28	Como administrador, quiero filtrar incidentes por categoría para analizar tipos específicos de problemas urbanos.	P0

Código	Historia de usuario	Prioridad
HU-29	Como administrador, quiero cambiar manualmente el estado de un incidente para reflejar su avance de atención.	P0
HU-30	Como administrador, quiero ver indicadores básicos para entender la situación general del sistema.	P0
HU-31	Como sistema, quiero advertir si un nuevo reporte parece duplicado de otro cercano para reducir registros repetidos.	P0
HU-32	Como ciudadano, quiero saber si ya existe un reporte similar cerca antes de enviar uno nuevo.	P1

Entregables obligatorios del Sprint 6

Entregable	Responsable principal	Estado esperado
Ruta/pantalla /admin/dashboard	Frontend Dashboard	Funcional
Endpoint GET /admin/incidents	Backend/API	Funcional
Endpoint PATCH /admin/incidents/{id}/status	Backend/API	Funcional
Endpoint GET /admin/metrics	Backend/API	Funcional
Filtros por estado y categoría	Frontend + Backend	Funcional
Tabla/lista de incidentes	Frontend Dashboard	Funcional
Tarjetas de indicadores básicos	Frontend Dashboard	Funcional
Detección básica de duplicados	GeoData/IA + Backend	Funcional
Advertencia de posible duplicado al reportar	Frontend PWA	Funcional
Pruebas de dashboard y duplicados	Scrum Master / QA	Ejecutadas
Historias aceptadas o rechazadas	Product Owner Técnico	Cerradas

Product Backlog seleccionado para Sprint 6

Código	Ítem	Prioridad	Responsable
PB-59	Crear pantalla /admin/dashboard	P0	Frontend PWA/Dashboard
PB-60	Crear endpoint GET /admin/incidents	P0	Backend/API
PB-61	Implementar filtros por estado	P0	Backend/API + Frontend
PB-62	Implementar filtros por categoría	P0	Backend/API + Frontend
PB-63	Crear endpoint para cambio de estado	P0	Backend/API
PB-64	Crear UI de cambio manual de estado	P0	Frontend Dashboard
PB-65	Crear endpoint de métricas básicas	P0	Backend/API
PB-66	Crear tarjetas de indicadores	P0	Frontend Dashboard
PB-67	Implementar detección de duplicados por radio de 50 m	P0	GeoData/IA
PB-68	Integrar advertencia de duplicado en /report	P1	Frontend PWA
PB-69	Probar flujo dashboard completo	P0	Scrum Master / QA
PB-70	Probar detección de duplicados	P0	Scrum Master / QA

Reglas técnicas del Sprint 6

Estados administrables

Pendiente
Confirmado
En revisión
Resuelto

Reglas de cambio de estado

1. Un administrador puede cambiar el estado manualmente.
2. El estado cambiado debe reflejarse en dashboard, mapa e historial.
3. Cada cambio debe registrar fecha de actualización.
4. No se debe permitir un estado inexistente.

Indicadores básicos del dashboard

```
total_reportes
reportes_pendientes
reportes_confirmados
reportes_en_revision
reportes_resueltos
categoria_mas_frecuente
```

Regla básica de duplicidad

Un reporte se considera posible duplicado si existe otro incidente que cumpla:

```
misma categoría
distancia <= 50 metros
estado activo: Pendiente, Confirmado o En revisión
```

Lunes — Diseño del dashboard, métricas y regla de duplicidad

Objetivo del día

Definir exactamente qué verá el administrador, qué filtros tendrá, qué indicadores se mostrarán y cómo funcionará la detección básica de duplicados.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar HU-26 a HU-32 y confirmar alcance obligatorio del sprint.	P0	Historias confirmadas.	Define el alcance semanal.
2	Definir columnas obligatorias de la tabla del dashboard.	P0	Categoría, estado, fecha, validaciones, ubicación, acción.	Guía Frontend y Backend.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
3	Definir filtros obligatorios: estado y categoría.	P0	Filtros aprobados.	Backend y Frontend los implementan.
4	Definir indicadores básicos del dashboard.	P0	Total, pendientes, confirmados, en revisión, resueltos y categoría más frecuente.	Backend crea métricas.
5	Definir estados permitidos para cambio manual.	P0	Pendiente, Confirmado, En revisión, Resuelto.	Backend valida estados.
6	Definir regla oficial de posible duplicado: misma categoría + 50 m + estado activo.	P0	Regla cerrada.	GeoData la implementa.
7	Definir mensaje de advertencia para duplicado: "Existe un reporte similar cerca".	P1	Texto aprobado.	Frontend lo usa.
8	Confirmar que no se implementará gestión de cuadrillas en este sprint.	P0	Alcance protegido.	Evita sobrecarga.
9	Confirmar que no se implementará login administrador avanzado.	P0	Admin puede ser simulado/controlado.	Reduce complejidad.
10	Actualizar criterios de aceptación del dashboard y duplicados.	P0	QA alineado.	Se prueba desde martes.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Crear caso de prueba para listar incidentes en dashboard.	P0	Prueba base definida.	Depende de GET /admin/incidents .
2	Crear caso de prueba para filtro por estado.	P0	Prueba de filtro definida.	Depende de backend/frontend.
3	Crear caso de prueba para filtro por categoría.	P0	Prueba de filtro definida.	Depende de backend/frontend.
4	Crear caso de prueba para cambio de estado válido.	P0	Prueba central definida.	Depende del endpoint PATCH.
5	Crear caso de prueba para cambio de estado inválido.	P0	Prueba de error definida.	Valida seguridad lógica.
6	Crear caso de prueba para actualización del mapa después del cambio de estado.	P0	Prueba de consistencia definida.	Conecta Sprint 4 y 6.
7	Crear caso de prueba para métricas básicas.	P0	Prueba de indicadores definida.	Depende de GET /admin/metrics .
8	Crear caso de prueba para posible duplicado dentro de 50 m.	P0	Prueba geo definida.	Depende de GeoData.
9	Crear caso de prueba para no duplicado fuera de 50 m.	P0	Prueba negativa definida.	Depende de GeoData.
10	Actualizar tablero Scrum con tareas del Sprint 6.	P0	Sprint Backlog asignado.	Control semanal.

Persona 3 — Developer Frontend PWA/Dashboard

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Crear ruta /admin/dashboard .	P0	Ruta visible.	Base del dashboard.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
2	Crear layout base del dashboard con encabezado, métricas y tabla.	P0	Estructura visual lista.	Depende de definición PO.
3	Crear componente <code>AdminMetricCard</code> .	P0	Tarjeta reutilizable preparada.	Se alimentará con métricas.
4	Crear componente <code>AdminIncidentTable</code> .	P0	Tabla base preparada.	Consumirá incidentes.
5	Crear componente <code>StatusFilter</code> .	P0	Filtro visual por estado listo.	Se conectará con API.
6	Crear componente <code>CategoryFilter</code> .	P0	Filtro visual por categoría listo.	Se conectará con API.
7	Crear componente <code>StatusSelect</code> para cambio manual de estado.	P0	Selector visual preparado.	Depende de estados permitidos.
8	Preparar estados frontend: loading, error, filters, incidents, metrics.	P0	Estado interno listo.	Facilita integración.
9	Revisar pantalla <code>/report</code> para ubicar advertencia de duplicado.	P1	Punto de integración definido.	Se usará miércoles/jueves.
10	Mantener dashboard separado de vistas ciudadanas.	P0	No confunde roles.	Criterio de producto.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Definir contrato <code>GET /admin/incidents</code> .	P0	Endpoint diseñado.	Frontend depende.
2	Definir parámetros de filtro: <code>status</code> y <code>category</code> .	P0	Query params definidos.	Filtros dependen de esto.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
3	Definir contrato PATCH /admin/incidents/{id}/status .	P0	Endpoint diseñado.	Cambio de estado depende.
4	Definir contrato GET /admin/metrics .	P0	Métricas diseñadas.	Frontend usa tarjetas.
5	Definir validación de estados permitidos.	P0	Estados inválidos serán rechazados.	QA lo probará.
6	Definir campos del listado admin: id, categoría, estado, fecha, validaciones, alias, ubicación.	P0	Response completa.	Tabla depende.
7	Definir si endpoint admin requiere token o modo simulado MVP.	P0	Seguridad mínima definida.	Alcance.
8	Preparar consulta para conteo por estado.	P0	Base de métricas.	Endpoint metrics.
9	Preparar consulta para categoría más frecuente.	P1	Métrica lista.	Indicador.
10	Documentar contratos iniciales.	P1	Swagger/README actualizado.	Integración.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar función de distancia usada en Sprint 5.	P0	Función reutilizable identificada.	Base de duplicados.
2	Confirmar radio de duplicidad: 50 metros.	P0	Parámetro definido.	Regla central.
3	Definir estados activos para duplicidad: Pendiente, Confirmado, En revisión.	P0	Estados cerrados.	Backend filtra.
4	Definir algoritmo: misma categoría + distancia <= 50 m + estado activo.	P0	Lógica documentada.	Se implementa martes.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
5	Preparar dataset con incidentes duplicados simulados.	P0	Casos listos para QA.	Pruebas de duplicidad.
6	Preparar dataset con incidentes cercanos pero diferente categoría.	P0	No debe marcar duplicado.	Caso negativo.
7	Preparar dataset con misma categoría pero fuera de 50 m.	P0	No debe marcar duplicado.	Caso negativo.
8	Definir respuesta esperada si hay duplicado: id, categoría, distancia, estado.	P1	Contrato para frontend/backend.	Advertencia.
9	Coordinar con Backend dónde se ejecutará la verificación: antes de crear reporte o endpoint separado.	P0	Decisión técnica cerrada.	Impacta /report .
10	Documentar regla de duplicidad.	P1	Trazabilidad técnica.	Cierre sprint.

Resultado obligatorio del lunes

Al cerrar lunes debe existir:

- historias del Sprint 6 confirmadas
 - columnas del dashboard definidas
 - filtros definidos
 - métricas definidas
 - estados administrables definidos
 - regla de duplicidad definida
 - componentes frontend planificados
 - contratos backend diseñados
 - casos QA preparados
-

Martes — Endpoints del dashboard y base visual funcional

Objetivo del día

Construir el backend del dashboard y dejar una primera versión visual conectable en frontend.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar primera estructura visual del dashboard.	P0	Layout aprobado o corregido.	Depende Frontend.
2	Validar que la tabla no muestre demasiada información.	P1	Tabla clara.	UX administrativo.
3	Validar que los filtros estén visibles y sean entendibles.	P0	Filtros aceptados.	Frontend.
4	Validar que los indicadores estén arriba de la tabla.	P0	Jerarquía visual aprobada.	Frontend.
5	Revisar response real de <code>GET /admin/incidents</code> .	P0	Datos suficientes.	Backend.
6	Revisar response real de <code>GET /admin/metrics</code> .	P0	Métricas suficientes.	Backend.
7	Confirmar que no se expongan datos sensibles en dashboard.	P0	Privacidad validada.	Backend/QA.
8	Revisar avance contra Sprint Goal.	P0	Riesgos detectados.	Control.
9	Repriorizar tareas si dashboard o duplicados se retrasan.	P0	P0 protegidos.	Gestión Scrum.
10	Actualizar criterios si alguna métrica cambia por limitación técnica.	P1	QA actualizado.	Evita pruebas falsas.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Probar GET /admin/incidents sin filtros.	P0	Devuelve lista general.	Backend.
2	Probar GET /admin/incidents? status=Pendiente .	P0	Filtra por estado.	Backend.
3	Probar GET /admin/incidents? category=Saneamiento .	P0	Filtra por categoría.	Backend.
4	Probar filtros combinados estado + categoría.	P0	Devuelve lista correcta.	Backend.
5	Probar GET /admin/metrics .	P0	Devuelve métricas.	Backend.
6	Verificar conteos de métricas contra datos reales en BD.	P0	Métricas confiables.	Crítico.
7	Probar que dashboard no muestre password_hash ni correos si no corresponde.	P0	Privacidad.	Seguridad.
8	Registrar bugs de endpoints admin.	P0	Bugs visibles.	Corrección miércoles.
9	Preparar prueba de cambio de estado para miércoles.	P0	Caso listo.	Endpoint PATCH.
10	Revisar tablero y bloquear tareas incompletas.	P0	Seguimiento real.	Scrum.

Persona 3 — Developer Frontend PWA/Dashboard

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar layout base de /admin/dashboard .	P0	Pantalla visible.	Base visual.
2	Conectar GET /admin/metrics con tarjetas superiores.	P0	Métricas visibles.	Depende Backend.
3	Conectar GET /admin/incidents con tabla.	P0	Tabla consume API.	Depende Backend.
4	Mostrar categoría, estado, fecha, validaciones y ubicación en tabla.	P0	Datos clave visibles.	PO lo valida.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
5	Implementar filtro visual por estado.	P0	Selector cambia estado local.	Luego llama API.
6	Implementar filtro visual por categoría.	P0	Selector cambia categoría local.	Luego llama API.
7	Crear función <code>fetchAdminIncidents(filters)</code> .	P0	Filtros consumen API.	Integración.
8	Crear estado de carga de dashboard.	P1	Loading visible.	UX.
9	Crear estado de error de dashboard.	P1	Error controlado.	QA.
10	Ajustar diseño responsive para laptop/proyector.	P1	Demo limpia.	Presentación.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar <code>GET /admin/incidents</code> .	P0	Endpoint funcional.	Dashboard.
2	Implementar filtro por estado.	P0	Query <code>status</code> funcional.	Filtro frontend.
3	Implementar filtro por categoría.	P0	Query <code>category</code> funcional.	Filtro frontend.
4	Implementar filtros combinados.	P0	Estado + categoría funcional.	QA lo prueba.
5	Incluir <code>validation_count</code> en listado admin.	P0	Tabla completa.	Depende Sprint 5.
6	Incluir coordenadas o ubicación resumida.	P0	Datos geográficos visibles.	Dashboard.
7	Implementar <code>GET /admin/metrics</code> .	P0	Métricas disponibles.	Tarjetas frontend.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
8	Calcular conteos por estado.	P0	Métricas correctas.	Endpoint metrics.
9	Calcular categoría más frecuente.	P1	Indicador adicional.	Dashboard.
10	Documentar endpoints admin iniciales.	P1	Swagger/README.	Integración.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar función <code>findPotentialDuplicates</code> .	P0	Lógica de duplicados lista.	Backend la integrará.
2	Probar duplicado por misma categoría dentro de 50 m.	P0	Detecta duplicado.	QA lo validará.
3	Probar no duplicado por distinta categoría dentro de 50 m.	P0	No marca duplicado.	Caso negativo.
4	Probar no duplicado por misma categoría fuera de 50 m.	P0	No marca duplicado.	Caso negativo.
5	Definir salida de duplicados: lista o primer duplicado más cercano.	P1	Respuesta definida.	Frontend.
6	Coordinar con Backend integración en creación de reporte o endpoint separado.	P0	Integración decidida.	Miércoles.
7	Preparar incidentes de prueba para dashboard con distintos estados.	P1	Métricas variadas.	Dashboard.
8	Validar que coordenadas del listado admin sean utilizables.	P0	Datos correctos.	Tabla.
9	Documentar pruebas de duplicidad realizadas.	P1	Evidencia.	Cierre.
10	Coordinar con QA casos exactos de duplicidad para jueves.	P0	Pruebas controladas.	QA.

Resultado obligatorio del martes

Al cerrar martes debe existir:

- dashboard visual base
 - tabla admin conectada o casi conectada
 - métricas básicas disponibles
 - endpoint `GET /admin/incidents`
 - endpoint `GET /admin/metrics`
 - filtros backend funcionando
 - función de duplicados implementada en base
 - pruebas iniciales de dashboard ejecutadas
-

Miércoles — Cambio manual de estado e integración de duplicados

Objetivo del día

Permitir que el administrador cambie el estado de un incidente y que el sistema pueda advertir duplicados durante el reporte ciudadano.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Validar flujo administrativo: ver incidente → cambiar estado → confirmar actualización.	P0	Flujo aprobado.	Depende Frontend/Backend.
2	Validar que los estados se actualicen en mapa e historial.	P0	Criterio obligatorio.	Conecta Sprint 4.
3	Definir mensaje al cambiar estado correctamente.	P1	Texto aprobado.	Frontend lo usa.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
4	Definir mensaje de error para estado inválido.	P1	Texto aprobado.	QA.
5	Validar cómo aparecerá la advertencia de duplicado en <code>/report</code> .	P1	Modal, banner o alerta definido.	Frontend.
6	Definir si el usuario puede enviar igual pese a duplicado.	P0	Recomendado: sí, pero advertido.	Regla de producto.
7	Validar que duplicados no bloqueen reportes automáticamente en MVP.	P0	Evita pérdida de reportes reales.	GeoData/Backend.
8	Revisar que el dashboard no permita borrar incidentes.	P0	Alcance protegido.	Evita funciones innecesarias.
9	Revisar avance contra Sprint Goal.	P0	Riesgos detectados.	Control.
10	Ajustar alcance si duplicados se vuelve demasiado costoso.	P0	Prioridad protegida.	Cierre del sprint.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Probar <code>PATCH /admin/incidents/{id}/status</code> con estado válido.	P0	Estado actualizado.	Backend.
2	Probar <code>PATCH</code> con estado inválido.	P0	Solicitud rechazada.	Validación.
3	Probar que el cambio se refleje en <code>GET /admin/incidents</code> .	P0	Dashboard actualizado.	Consistencia.
4	Probar que el cambio se refleje en <code>GET /incidents/public</code> .	P0	Mapa actualizado.	Consistencia.
5	Probar que el cambio se refleje en <code>/my-reports</code> .	P0	Historial actualizado.	Consistencia.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
6	Probar duplicado dentro de 50 m misma categoría.	P0	Advertencia aparece.	GeoData/Frontend.
7	Probar misma ubicación pero categoría distinta.	P0	No debe advertir duplicado.	Regla.
8	Probar misma categoría pero fuera de 50 m.	P0	No debe advertir duplicado.	Regla.
9	Registrar bugs de cambio de estado y duplicados.	P0	Bugs visibles.	Corrección jueves.
10	Preparar prueba completa dashboard + duplicado para viernes.	P1	Guion listo.	Review.

Persona 3 — Developer Frontend PWA/Dashboard

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Conectar StatusSelect con PATCH /admin/incidents/{id}/status .	P0	Cambio de estado desde UI.	Depende Backend.
2	Actualizar fila de la tabla después del cambio de estado.	P0	Dashboard se refresca.	UX.
3	Mostrar mensaje de éxito al cambiar estado.	P1	Feedback claro.	PO lo valida.
4	Mostrar error si el cambio falla.	P0	Error controlado.	QA.
5	Refrescar métricas después de cambio de estado.	P0	Indicadores consistentes.	Importante.
6	Aplicar filtros en frontend llamando a API filtrada.	P0	Filtros funcionales.	Dashboard.
7	Integrar advertencia visual de posible duplicado en /report .	P1	Banner/modal aparece.	Depende Backend/GeoData.
8	Permitir continuar envío pese a advertencia de duplicado.	P1	Usuario decide.	Regla PO.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
9	Corregir errores visuales de tabla y filtros.	P0	Dashboard estable.	QA.
10	Verificar dashboard en pantalla de presentación.	P1	Visual claro.	Demo.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar PATCH <code>/admin/incidents/{id}/status</code> .	P0	Endpoint funcional.	Cambio manual.
2	Validar estado recibido contra lista permitida.	P0	Estados inválidos rechazados.	QA.
3	Actualizar campo <code>status</code> del incidente.	P0	Estado persistido.	Dashboard/mapa/historial
4	Actualizar campo <code>updated_at</code> si existe.	P1	Trazabilidad básica.	Mantenibilidad.
5	Devolver incidente actualizado en respuesta.	P0	Frontend actualiza UI.	Integración.
6	Ajustar endpoints públicos para reflejar estado actualizado.	P0	Consistencia global.	Sprint 4 conectado.
7	Integrar función de duplicados antes de crear reporte o mediante endpoint de verificación.	P0	Duplicados operativos.	GeoData.
8	Si se usa endpoint separado, crear <code>POST /incidents/check-duplicate</code> .	P1	Verificación previa.	Frontend report.
9	Si se integra en <code>POST /incidents</code> , devolver advertencia <code>possible_duplicate</code> .	P1	Reporte advierte duplicado.	Frontend.
10	Documentar endpoints de cambio de estado y duplicados.	P1	API clara.	Cierre técnico.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Integrar <code>findPotentialDuplicates</code> con Backend.	P0	Duplicados conectados.	Endpoint o POST report.
2	Verificar que solo considere estados activos.	P0	Resueltos no cuentan como duplicado.	Regla correcta.
3	Verificar que compare misma categoría.	P0	Categoría correcta.	Regla central.
4	Verificar distancia ≤ 50 m.	P0	Radio correcto.	Regla central.
5	Calcular distancia al duplicado más cercano.	P1	Se puede mostrar al usuario.	Frontend.
6	Preparar respuesta con datos mínimos del duplicado encontrado.	P1	id, categoría, estado, distancia.	Frontend.
7	Probar duplicado antes de registrar nuevo reporte.	P0	Advertencia correcta.	QA.
8	Probar no duplicado con dataset negativo.	P0	Sin advertencia.	QA.
9	Documentar limitaciones: no usa similitud de imagen ni texto en MVP.	P1	Alcance claro.	Presentación.
10	Coordinar con PO si la duplicidad bloquea o solo advierte.	P0	Regla final implementada.	Producto.

Resultado obligatorio del miércoles

Al cerrar miércoles debe existir:

- endpoint de cambio de estado funcional
- UI de cambio de estado conectada
- estado actualizado visible en dashboard

- métricas refrescables
 - duplicados integrados o endpoint de verificación creado
 - advertencia de duplicado en avance
 - pruebas de cambio de estado iniciadas
-

Jueves — Estabilización, consistencia entre vistas y pruebas completas

Objetivo del día

Corregir errores y asegurar que dashboard, mapa, historial y duplicados se mantengan consistentes.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar dashboard completo con datos reales.	P0	Dashboard aceptable.	Depende integración.
2	Validar que filtros funcionen desde la perspectiva del administrador.	P0	Filtros útiles.	QA.
3	Validar que métricas comuniquen valor institucional.	P0	Indicadores aprobados.	Demo.
4	Validar que cambio de estado no sea confuso.	P0	Flujo administrativo claro.	Frontend.
5	Validar advertencia de duplicado en reporte.	P1	Advertencia clara.	Frontend/GeoData.
6	Confirmar que el usuario pueda continuar pese a duplicado.	P0	Regla aceptada.	Producto.
7	Revisar que no haya datos privados en dashboard.	P0	Privacidad aceptada.	Seguridad.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
8	Definir qué queda fuera para Sprint 7 si algo no cierra.	P0	Alcance controlado.	Cierre.
9	Preparar criterios de aceptación final del Sprint 6.	P0	Viernes listo.	PO.
10	Definir entrada del Sprint 7: integración, pruebas y demo final.	P1	Continuidad.	Último sprint.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Ejecutar flujo admin: abrir dashboard → filtrar por estado.	P0	Filtro validado.	Cierre.
2	Ejecutar flujo admin: filtrar por categoría.	P0	Filtro validado.	Cierre.
3	Ejecutar flujo admin: cambiar estado Pendiente → En revisión.	P0	Cambio validado.	Central.
4	Ejecutar flujo admin: cambiar En revisión → Resuelto.	P0	Cambio validado.	Central.
5	Verificar que el mapa refleje el nuevo estado.	P0	Consistencia.	Sprint 4.
6	Verificar que historial refleje el nuevo estado.	P0	Consistencia.	Sprint 4.
7	Verificar que métricas cambien después del cambio de estado.	P0	Consistencia.	Dashboard.
8	Ejecutar prueba de duplicado positiva.	P0	Advertencia aparece.	Duplicados.
9	Ejecutar prueba de duplicado negativa.	P0	Advertencia no aparece.	Duplicados.
10	Clasificar bugs críticos, medios y menores.	P0	Corrección ordenada.	Viernes.
11	Actualizar evidencias de QA en tablero.	P1	Trazabilidad.	Review.

Persona 3 — Developer Frontend PWA/Dashboard

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir bugs críticos de tabla admin.	P0	Tabla estable.	QA.
2	Corregir bugs críticos de filtros.	P0	Filtros estables.	QA.
3	Corregir bugs críticos de cambio de estado.	P0	Selector estable.	QA.
4	Corregir refresco de métricas después de cambios.	P0	Métricas consistentes.	QA.
5	Corregir visualización de estados en dashboard.	P0	Estados claros.	PO.
6	Corregir advertencia de duplicado en /report .	P1	Advertencia estable.	QA.
7	Corregir errores de navegación entre dashboard, mapa e historial.	P0	Flujo estable.	Demo.
8	Mejorar diseño visual del dashboard para presentación.	P1	Más impacto visual.	Demo.
9	Verificar funcionamiento en navegador de escritorio y móvil.	P0	Compatibilidad básica.	Review.
10	Preparar capturas o demo corta del dashboard.	P1	Evidencia.	Review.
11	Congelar cambios visuales no críticos al final del día.	P1	Evita romper sprint.	Cierre.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir bugs de GET /admin/incidents .	P0	Listado estable.	QA.
2	Corregir bugs de filtros backend.	P0	Filtros exactos.	QA.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
3	Corregir bugs de PATCH <code>/admin/incidents/{id}/status</code> .	P0	Cambio estable.	QA.
4	Corregir bugs de GET <code>/admin/metrics</code> .	P0	Métricas exactas.	QA.
5	Verificar que el cambio de estado actualice <code>updated_at</code> .	P1	Trazabilidad básica.	Calidad.
6	Verificar que endpoints públicos reflejen estado actualizado.	P0	Consistencia.	Mapa/historial.
7	Corregir bugs de duplicados.	P0	Regla estable.	GeoData/QA.
8	Revisar que reportes resueltos no se marquen como duplicados activos.	P0	Regla correcta.	Duplicados.
9	Revisar que no se expongan datos sensibles en endpoints admin.	P0	Seguridad.	PO/QA.
10	Actualizar documentación API con cambios finales.	P1	Cierre técnico.	Review.
11	Preparar datos para demo final del dashboard.	P1	Dataset útil.	Presentación.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir errores en cálculo de duplicidad.	P0	Duplicados exactos.	QA.
2	Verificar regla misma categoría + 50 m.	P0	Regla central validada.	QA.
3	Verificar que diferentes categorías no generen advertencia.	P0	Caso negativo validado.	QA.
4	Verificar que reportes fuera de 50 m no generen advertencia.	P0	Caso negativo validado.	QA.
5	Verificar que reportes resueltos no bloqueen ni adviertan duplicado activo.	P0	Estado activo correcto.	Backend.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
6	Preparar dataset final para demo de duplicados.	P1	Prueba controlada.	Review.
7	Coordinar con Frontend mensaje de distancia aproximada si se muestra.	P1	Advertencia más clara.	UX.
8	Coordinar con Backend respuesta final de duplicado.	P0	Formato estable.	Integración.
9	Documentar reglas finales de duplicidad.	P1	Cierre técnico.	Review.
10	Preparar entrada para Sprint 7: pruebas integrales de geodatos.	P1	Continuidad.	Último sprint.
11	Validar que los datos del dashboard tengan coordenadas limpias.	P0	Mapa y admin consistentes.	Demo.

Resultado obligatorio del jueves

Al cerrar jueves debe existir:

- dashboard estable
 - filtros funcionales
 - cambio manual de estado funcional
 - métricas actualizadas
 - duplicados detectados correctamente
 - advertencia de duplicado funcional o casi cerrada
 - consistencia entre dashboard, mapa e historial
 - bugs críticos en corrección final
-

Viernes — Pruebas finales, Sprint Review y cierre

Objetivo del día

Cerrar Sprint 6 demostrando un dashboard funcional y una detección básica de duplicados.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar HU-26: lista de incidentes en dashboard.	P0	Historia aceptada o rechazada.	Cierre.
2	Revisar HU-27: filtro por estado.	P0	Historia aceptada o rechazada.	Cierre.
3	Revisar HU-28: filtro por categoría.	P0	Historia aceptada o rechazada.	Cierre.
4	Revisar HU-29: cambio manual de estado.	P0	Historia aceptada o rechazada.	Cierre.
5	Revisar HU-30: indicadores básicos.	P0	Historia aceptada o rechazada.	Cierre.
6	Revisar HU-31: detección básica de duplicados.	P0	Historia aceptada o rechazada.	Cierre.
7	Revisar HU-32: advertencia al ciudadano si se completó.	P1	Aceptada o trasladada.	Control.
8	Validar que el dashboard comunique valor municipal.	P0	Valor institucional aprobado.	Narrativa.
9	Definir exactamente qué se pulirá en Sprint 7.	P0	Último sprint preparado.	Continuidad.
10	Aprobar o rechazar incremento completo del Sprint 6.	P0	Cierre formal.	Scrum Review.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Ejecutar prueba final: abrir dashboard y listar incidentes.	P0	Dashboard validado.	Cierre.
2	Ejecutar prueba final: filtrar por estado.	P0	Filtro validado.	Cierre.
3	Ejecutar prueba final: filtrar por categoría.	P0	Filtro validado.	Cierre.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
4	Ejecutar prueba final: cambiar estado de incidente.	P0	Cambio validado.	Cierre.
5	Verificar que estado actualizado aparezca en mapa.	P0	Consistencia.	Cierre.
6	Verificar que estado actualizado aparezca en historial.	P0	Consistencia.	Cierre.
7	Verificar métricas antes y después de cambio de estado.	P0	Indicadores confiables.	Cierre.
8	Ejecutar prueba final de duplicado positivo.	P0	Advertencia validada.	Cierre.
9	Ejecutar prueba final de duplicado negativo.	P0	Sin falsa advertencia.	Cierre.
10	Registrar bugs finales no bloqueantes para Sprint 7.	P0	Deuda controlada.	Próximo sprint.
11	Realizar retrospectiva del Sprint 6.	P1	Mejoras registradas.	Último sprint.

Persona 3 — Developer Frontend PWA/Dashboard

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir bugs críticos restantes del dashboard.	P0	Dashboard estable.	QA.
2	Corregir bugs críticos restantes de filtros.	P0	Filtros estables.	QA.
3	Corregir bugs críticos restantes del cambio de estado.	P0	Selector estable.	QA.
4	Corregir bugs críticos restantes de métricas.	P0	Indicadores visibles.	QA.
5	Corregir bugs críticos de advertencia de duplicado.	P1	Advertencia estable si entra.	QA.
6	Verificar dashboard en pantalla de presentación.	P1	Buena legibilidad.	Demo.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
7	Preparar demo visual del Sprint 6.	P1	Guion visual listo.	Review.
8	Actualizar README frontend con dashboard.	P1	Documentación mínima.	Cierre.
9	Congelar dashboard para no romperlo antes del Review.	P0	Versión estable.	Control.
10	Preparar listado de mejoras visuales para Sprint 7.	P1	Pulido posterior.	Último sprint.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir bugs finales de endpoints admin.	P0	API estable.	QA.
2	Corregir bugs finales de cambio de estado.	P0	PATCH estable.	QA.
3	Corregir bugs finales de métricas.	P0	Indicadores confiables.	QA.
4	Corregir bugs finales de duplicados.	P0	Detección estable.	QA.
5	Verificar consistencia de estado en todos los endpoints.	P0	Datos coherentes.	Cierre.
6	Verificar que estados inválidos sean rechazados.	P0	Integridad.	Cierre.
7	Verificar que no haya datos sensibles en respuestas finales.	P0	Seguridad.	Cierre.
8	Actualizar README/Swagger final del Sprint 6.	P1	Documentación.	Cierre técnico.
9	Preparar base de datos limpia para demo del Sprint 7.	P1	Datos demo listos.	Último sprint.
10	Registrar deuda técnica para Sprint 7.	P1	Pendientes controlados.	Continuidad.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Verificar duplicado positivo con dataset final.	P0	Advertencia correcta.	Cierre.
2	Verificar duplicado negativo por distancia.	P0	Sin advertencia.	Cierre.
3	Verificar duplicado negativo por categoría distinta.	P0	Sin advertencia.	Cierre.
4	Verificar duplicado negativo por estado Resuelto.	P0	Sin advertencia activa.	Cierre.
5	Corregir último error de cálculo si aparece.	P0	Duplicidad estable.	QA.
6	Documentar reglas finales de duplicidad.	P1	Evidencia técnica.	Review.
7	Preparar casos de prueba geográficos para Sprint 7.	P1	Pruebas integrales listas.	Último sprint.
8	Coordinar con Backend datos finales de dashboard.	P1	Datos limpios.	Demo.
9	Validar que no haya coordenadas nulas en datos de demo.	P0	Demo segura.	Presentación.
10	Registrar limitaciones de duplicidad: no compara imagen ni texto.	P1	Alcance transparente.	Narrativa.

Resultado obligatorio del viernes

Al cerrar viernes, el equipo debe poder demostrar:

1. El administrador abre `/admin/dashboard`.
2. Ve todos los incidentes registrados.
3. Filtra incidentes por estado.
4. Filtra incidentes por categoría.

5. Ve indicadores básicos del sistema.
 6. Cambia manualmente un incidente de **Pendiente** a **En revisión**.
 7. El cambio se refleja en dashboard.
 8. El cambio se refleja en mapa ciudadano.
 9. El cambio se refleja en historial del ciudadano.
 10. Un ciudadano intenta crear un reporte similar dentro de 50 m.
 11. El sistema advierte posible duplicado.
 12. Si el reporte es de otra categoría o está fuera de 50 m, no advierte duplicado.
-

Definition of Done del Sprint 6

Criterio	Obligatorio
Dashboard inicial existe	Sí
GET /admin/incidents funciona	Sí
Filtro por estado funciona	Sí
Filtro por categoría funciona	Sí
PATCH /admin/incidents/{id}/status funciona	Sí
Estados inválidos son rechazados	Sí
Estado actualizado se refleja en dashboard	Sí
Estado actualizado se refleja en mapa	Sí
Estado actualizado se refleja en historial	Sí
GET /admin/metrics funciona	Sí
Métricas básicas son correctas	Sí
Detección de duplicados por 50 m funciona	Sí
Duplicado considera misma categoría	Sí
Duplicado ignora reportes resueltos	Sí
QA ejecutó pruebas de dashboard y duplicados	Sí
Bugs críticos corregidos	Sí
Sprint Review realizado	Sí

Incremento esperado al final del Sprint 6

QHALI MVP - Incremento Sprint 6

- └─ /admin/dashboard
- └─ GET /admin/incidents
- └─ Filtro por estado
- └─ Filtro por categoría
- └─ PATCH /admin/incidents/{id}/status
- └─ GET /admin/metrics
- └─ Tarjetas de indicadores
- └─ Tabla administrativa de incidentes
- └─ Cambio manual de estado
- └─ Consistencia dashboard/mapa/historial
- └─ Detección básica de duplicados
- └─ Advertencia de posible duplicado
- └─ Pruebas funcionales del dashboard y duplicidad

Sprint Review del Sprint 6

La demo debe seguir este guion:

1. Iniciar sesión como administrador o entrar al modo admin del MVP.
2. Abrir /admin/dashboard.
3. Mostrar indicadores:
 - total de reportes
 - pendientes
 - confirmados
 - en revisión
 - resueltos
 - categoría más frecuente
4. Filtrar por estado: Pendiente.
5. Filtrar por categoría: Infraestructura vial.
6. Cambiar un incidente de Pendiente a En revisión.
7. Ir al mapa ciudadano y verificar cambio de color/estado.
8. Ir al historial del ciudadano y verificar cambio de estado.
9. Crear un reporte similar dentro de 50 m.
10. Mostrar advertencia: “Existe un reporte similar cerca”.
11. Crear otro reporte de categoría distinta o fuera de 50 m.
12. Verificar que no aparezca advertencia.

Retrospectiva del Sprint 6

Preguntas mínimas:

1. ¿Qué fue más difícil: dashboard, métricas, cambio de estado o duplicados?
 2. ¿El dashboard comunica valor institucional o aún parece muy simple?
 3. ¿Los filtros ayudan realmente a gestionar incidentes?
 4. ¿El cambio de estado mantiene consistencia entre vistas?
 5. ¿La detección de duplicados reduce reportes repetidos sin bloquear reportes reales?
 6. ¿Qué se debe pulir obligatoriamente en el Sprint 7 antes de la demo final?
-

Entrada directa para Sprint 7

Para empezar el Sprint 7 deben quedar listos:

1. Ciudadano con autenticación funcional.
2. Reporte con foto, categoría, descripción y GPS.
3. Mapa ciudadano funcional.
4. Historial privado funcional.
5. Validación ciudadana cercana funcional.
6. Cambio automático a Confirmado con 5 validaciones.
7. Dashboard administrativo funcional.
8. Filtros e indicadores básicos.
9. Cambio manual de estado.
10. Detección básica de duplicados.

El **Sprint 7** será el cierre del MVP:

Integración completa
Pruebas de punta a punta
Corrección de bugs críticos
Pulido visual
Datos de demo
Guion de presentación
Sprint Review final